2ee7d85c-1c52-4378-a0d9-f92cde57c195

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'soybeanjs/soybean-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/soybean-ui/soybean-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-10T06:06:19.834Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Refactor `STable` aggregation into headless `DataTableRoot`
</pr_title>

<pr_description>
This PR starts the table refactor by separating structural table primitives from the aggregated data-table behavior. The existing `STable` API remains the entrypoint, but the row/column rendering, selection, expansion, and slot orchestration now live in headless so later work can build on a single non-styled core.

- **What changed**
  - Added a new headless `DataTableRoot` that owns:
    - column filtering

    - cell value resolution by `dataIndex`
    - single/multi row selection state
    - expandable row state
    - header/body/footer rendering over one `<table>`
    - typed slot contracts for header, cell, selection, expand, and footer customization
  - Kept existing headless `Table*` components as pure structural primitives.

- **UI layer simplification**
  - Refactored `/src/components/table/table.vue` into a thin wrapper around `DataTableRoot`.
  - UI now focuses on:
    - variant/style injection via `provideTableUi`
    - default visual implementations for selection and expand controls
    - forwarding consumer slots to the headless core

- **API compatibility**
  - Preserved the current `STable` surface for:
    - column definitions
    - `selected` / `expanded` controlled state
    - selection and expandable rows
    - footer slot usage
    - existing dynamic header/cell slot patterns
  - Re-exported the new headless data-table types so downstream typing continues to work from the table package entrypoints.

- **Accessibility adjustments**
  - Added clearer built-in row labels for selection controls.
  - Disabled header selection when no rows are present.
  - Kept default selection/expand affordances overrideable through slots.

- **Foundation for next phases**
  - Introduces a dedicated headless aggregation layer without mixing it into the primitive `Table*` namespace.
  - Creates a better base for upcoming work on grouped headers, fixed columns, sorting/filtering, tree rows, and virtualization.

```vue
<STable :columns="columns" :data="data" :row-key="row => row.id">
  <template #age="{ value }">
    <span class="font-medium">Age: {{ value }}</span>
  </template>
```

```
  <template #expanded-row="{ row }">
    <div class="p-2">Expanded {{ row.name }}</div>
  </template>
</STable>
```
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
Pull request overview

Refactors the existing `STable` implementation by introducing a new headless `DataTableRoot` that owns data-table aggregation behavior (selection/expansion/value resolution/render orchestration), while keeping the UI-layer `STable` as a thin styled wrapper.

Changes:
- Added headless `DataTableRoot` (+ types/shared helpers) and exposed it via headless root + namespaced exports.
- Refactored `src/components/table/table.vue` to wrap `DataTableRoot`, providing default selection/expand UI and forwarding consumer slots/listeners.
- Updated table package types/exports and added initial `STable` integration tests (rendering, slot forwarding, selection/expand emits, a11y).

Reviewed changes

Copilot reviewed 13 out of 13 changed files in this pull request and generated 3 comments.

💡 <a>Add Copilot custom instructions for smarter, more guided reviews. Learn how to get started.
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>headless/src/components/data-table/data-table-root.vue:184-191</file>
<original_commit>8c718ac</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
The built-in header selection checkbox does not visually reflect the indeterminate state. When `headerSelection === 'indeterminate'`, only `aria-checked="mixed"` is set, but the checkbox’s `indeterminate` DOM property is never set, so users won’t see a partial-selection indicator. Bind the `indeterminate` property (and keep `checked` in sync) so the default fallback UI matches the computed `CheckedState`.
</comment_old>

</comment_thread>

<comment_thread>
<file>headless/src/components/data-table/data-table-root.vue:253</file>
<original_commit>8c718ac</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
For single-selection mode (`multiple === false`), the default `<input type="radio">` uses `@change` to call `onSelectedChange(...)`. A radio input won’t emit `change` when clicking an already-selected option, so `useSelection`’s “clearable single select” behavior (toggling to `undefined`) can’t be triggered via the UI. Consider switching to `@click` (or a `<button>` implementation) so clicking an already-selected row can clear it when `singleClearable` is true, and optionally add a shared `name` to enable proper radio-group keyboard navigation.
```suggestion
                    @click="onSelectedChange(rowKey(item))"
```

```
</comment_old>

</comment_thread>

<comment_thread>
<file>src/components/table/table-radio.vue:23</file>
<original_commit>8c718ac</original_commit>

<comment_old>
<author>@copilot-pull-request-reviewer</author>
`TableRadio` now sets `role="radio"`/`aria-checked`, but it’s rendered inside table cells without an enclosing `role="radiogroup"` and without the expected roving-tabindex/arrow-key handling for ARIA radios. This can introduce accessibility violations (e.g., “ARIA required parent role”) and confusing keyboard behavior. Either wrap the set of radios in an appropriate `radiogroup` (and manage keyboard interaction), or avoid `role="radio"` here and use a button semantics pattern (e.g., `aria-pressed`) that matches the implemented behavior.
```suggestion
  <button type="button" :aria-
pressed="checked" :class="ui.radioRoot">
```
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>4221350638</comment_id>
<author>@honghuangdc</author>
@copilot apply changes based on the comments in this thread
</comment_new>

</pr_comments>

</comments>

The last **2** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
```

* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<prev_assistant>

Sorry, you've hit a rate limit that restricts the number of Copilot model requests you can make within a specific time period. Please try again in <duration>. Please review our Terms of Service (https://docs.github.com/site-policy/github-terms/github-terms-of-service). If the problem persists, please contact GitHub Support, including the request ID `B800:E7FEC:DF2210:100EB80:69D89ACF`. To retry, leave a comment on this pull request asking Copilot to try again.

<current>

<request>

You are working on a task in the 'soybeanjs/soybean-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/soybean-ui/soybean-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-10T10:18:56.895Z</current_datetime>

Consider the following problem statement:

<problem_statement>
继续处理
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.